Universität Bonn

Mitschrift zur Vorlesung

Grundlagen der algorithmischen Geometrie

gehalten von

Dr. Herman Haverkort,
PD. Dr. Elmar Langetepe

geTEXed von

Daniel Emse

Sommer 2026

Anmerkungen und Korrekturen

Falls du Anmerkungen/Korrekturen hast, melde dich bei mir (`daniel.emse@uni-bonn.de`).

Inhaltsverzeichnis

Vorlesung **1**

2 Das Sweep-Verfahren **2**
2.1 Einführung 2
2.2 Sweep im Eindimensionalen 2
2.2.1 Das Maximum einer Menge von Objekten 2
2.2.2 Das dichteste Paar in einer Menge von Zahlen 2
2.2.3 Die maximale Teilsumme 3
2.3 Sweep in der Ebene 3

Anhang **A**

Liste der Definitionen **B**

Liste der Aussagen **C**

Liste der Algorithmen **D**

Liste der Abkürzungen **E**

Index **F**

Vorlesung

Kapitel 2

Das Sweep-Verfahren

2.1 Einführung

[14.04.2026, Vorlesung 1]

Das *Sweep*-Verfahren ist ähnlich vielseitig wie *Divide and Conquer*, also ein hilfreiches Pardigma. Divide and Conquer ist beispielsweise sinnvoll, wenn die erwarteten Kosten des Ergebnisses klein sind. Beispielsweise erhalten wir $T(n) \leq 2 \cdot T(n/2) + c(\text{merge})$.

Sweep ist ein geometrisches Verfahren, bei dem wir lokal eine Dimensionsreduktion von einem n-dimensionalen Problem auf ein $(n-1)$-dimensionales Problem verwandeln, indem wir die Ebene mit einer *Sweepline* ausfegen, indem wir gemäß einer vorher angefertigten Sortierung fegen. Dabei folgt die Korrektheit des Verfahrens über eine Invariante nahe der Sweepline.

2.2 Sweep im Eindimensionalen

2.2.1 Das Maximum einer Menge von Objekten

Problem 2.1: Find maximum

Input: $Q = [q_1, \ldots, q_n] \in \mathbb{R}^n$
Output: $q_i \in Q$ mit $q_i \geq q_j \forall j = 1, \ldots, n$

Hierfür haben wir folgenden optimalen Algorithmus mit Laufzeit $\Theta(n)$. Dabei sweepen wir über die Indizes, erhalten also lokal ein 0-dimensionales Problem

Algorithmus 2.2 FindMax

Input: Wie in Problem 2.1
Output: Wie in Problem 2.1

```
1: MaxSoFar ← q[1]
2: for j = 2 to n do do
3:     if MaxSoFar < q[j] then
4:         MaxSoFar ← q[j]
5: return MaxSoFar
```

2.2.2 Das dichteste Paar in einer Menge von Zahlen

Problem 2.3: Closest pair in einer Dimension

Input: $x_1, \ldots, x_n \in \mathbb{R}$
Output: $i \neq j \in \{1, \ldots, n\}$ sodass $|x_i - x_j| = \min_{\ell \neq k} |x_\ell - x_k|$ für $\ell, k \in \{1, \ldots, n\}$

Wir sortieren die Zahlen aufsteigend, sodass wir ein neues Array $x'_1 \leq \cdots \leq x'_n \in \mathbb{R}$ bekommen (im Mehrdimensionalen sortieren wir sie nach einer bestimmten Koordinate). Das braucht aber natürlich bereits $\mathcal{O}(n \log(n))$ Zeit.

Algorithmus 2.4 FINDCLOSESTPAIR-1D

Input: Wie in Problem 2.3, aber als aufsteigend sortiertes Array x'
Output: Wie in Problem 2.3

```
1: MinDistSoFar ← x'[2] − x'[1]
2: ClosPos ← 2
3: for j = 3 to n do do
4:     if MinDistSoFar > x'[j] − x'[j − 1] then
5:         MinDistSoFar ← x'[j] − x'[j − 1]
6:         ClosPos ← j
7: return x'[ClosPos − 1], x'[ClosPos]
```

Korollar 2.5. *Die Berechnung des* CLOSEST PAIRS *von n reellen Zahlen benötigt Laufzeit $\mathcal{O}(n \log(n))$.*

2.2.3 Die maximale Teilsumme

Problem 2.6: Maximum subvector

Input: Array $V \in \mathbb{R}^n$
Output: Finde $i \leq j$, sodass $V[i] + V[i+1] + \cdots + V[j] = \max_{\ell \leq k} V[\ell] + \cdots + V[k]$, mit $i, j, \ell, k \in \{1, \ldots, n\}$

Algorithmus 2.7 FINDMAXIMUMSUBVECTOR

```
1: MaxSoFar ← 0
2: MaxEndingHere ← 0
3: for j = 1 to n do do
4:     MaxEndingHere ← max(0, MaxEndingHere + V[j])
5:     MaxSoFar ← max(MaxSoFar, MaxEndingHere)
6: return MaxSoFar
```

Satz 2.8: FindMaximumSubvector

In einer Folge von n Zahlen die maximale Teilsumme konsekutiver Zahlen zu bestimmen, hat die Zeitkomplexität $\Theta(n)$.

2.3 Sweep in der Ebene

Im Allgemeinen ist $|x| \coloneqq \|x\|_2$ die Euklidische Norm des Vektor x.

Problem 2.9: Closest pair in zwei Dimensionen

Input: $x_1, \ldots, x_n \in \mathbb{R}^2$
Output: $i \neq j \in \{1, \ldots, n\}$ sodass $|x_i - x_j| = \min_{\ell \neq k} |x_\ell - x_k|$ für $\ell, k \in \{1, \ldots, n\}$

Wir führen zum Verwalten ein Verzeichnis ein, in dem zu jedem Zeitpunkt den Status der *Sweep Line* tracked. Dieses heißt *Sweep-Status-Struktur (SSS)*.

Zu spezifizieren ist noch MinDist. Das folgt aber nächste Vorlesung. Dann erhalten wir folgenden Algorithmus:

Algorithmus 2.10 FindClosestPair-2D

Input: Array von n Punkten aus $\mathbb{R}^2$
Output: Zwei Elemente des Arrays mit dem geringsten euklidischen Abstand

```
1: Sortiere die n Punkte nach aufsteigenden x-Koordinaten und füge sie ins Array P ein
2: Füge P[1], P[2] in SSS ein
3: MinSoFar ← |P[1] − P[2]|
4: links ← 1
5: rechts ← 3
6: while rechts ≤ n do
7:     if P[links].x + MinSoFar ≤ P[rechts].x then
8:         Entferne P[links] aus SSS
9:         links ← links + 1
10:    else
11:        MinSoFar ← MinDist(SSS, P[rechts], MinSoFar)
12:        Füge P[rechts] in SSS ein
13:        rechts ← rechts + 1
14: return MinSoFar
```

[14.04.2026, Vorlesung 1]
[16.04.2026, Vorlesung 2]

[16.04.2026, Vorlesung 2]
[21.04.2026, Vorlesung 3]

Anhang

Liste der Definitionen

2.1 Find maximum 2
2.3 Closest pair in einer Dimension 2
2.6 Maximum subvector 3
2.9 Closest pair in zwei Dimensionen 3

Liste der Aussagen

2.8 FindMaximumSubvector 3

Liste der Algorithmen

2.2 FindMax 2
2.4 FindClosestPair-1D 3
2.7 FindMaximumSubvector 3
2.10 FindClosestPair-2D 4

Liste der Abkürzungen

SSS Sweep-Status-Struktur 3

Index

SSS, 3

Sweep, 2